MINI PROJECT ON

Automatic Railway Level Crossing Controller

Prepared By: Shashaank Malviya

Mini Project: Automatic Railway Level Crossing Controller

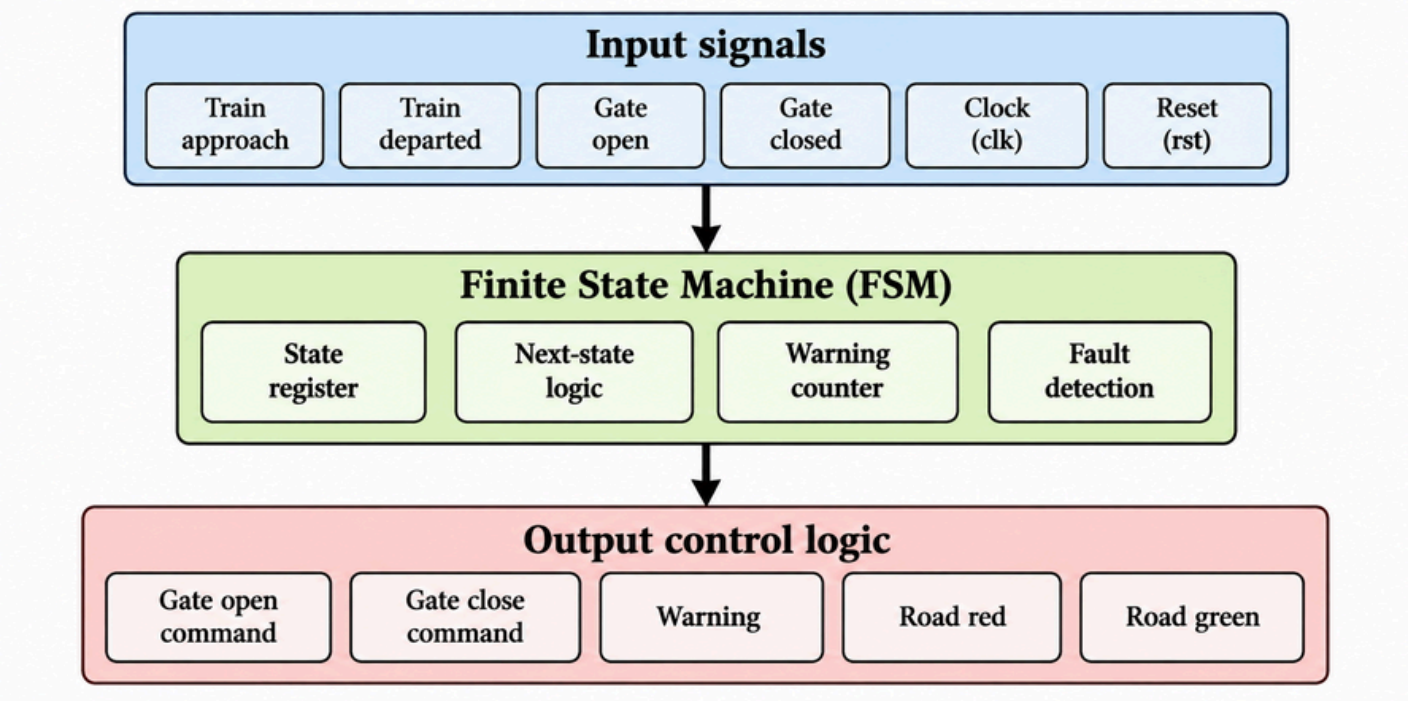
• Problem Statement:

The objective is to design a digital controller that manages the sequence of railway crossing operations based on train approach, train departure, and gate-position inputs. The controller must generate appropriate gate commands, warning signals, and road traffic signals. It must also detect selected invalid sensor combinations and enter a fault state.
The design is evaluated through functional simulation, code coverage analysis, and logic synthesis.

• Objectives:

- Design an FSM-based railway crossing controller.
- Implement the controller using Verilog RTL.
- Coordinate gate commands, warning indication, and road signals.
- Detect selected invalid gate-sensor combinations.
- Verify normal and fault operation using VCS and GTKWave.
- Evaluate code coverage using VCS.
- Synthesize the RTL using Design Compiler and analyze the generated reports.

• Architecture Block Diagram:



- **Input and Output Signals:**

Signal	Direction	Function
clk	Input	Synchronizes state transitions
rst	Input	Resets the FSM
train_approach	Input	Indicates an approaching train
train_departed	Input	Indicates train departure
gate_open	Input	Gate-open feedback
gate_closed	Input	Gate-closed feedback
gate_open_cmd	Output	Requests gate opening
gate_close_cmd	Output	Requests gate closing
warning	Output	Warning indication
road_red	Output	Stops road traffic
road_green	Output	Permits road traffic in the normal state

- **FSM State Coding:**

State	Encoding	Description
IDLE	000	Normal state, gate open
WARNING_ST	001	Warning interval before closing
CLOSING	010	Gate closing sequence
TRAIN_PASS	011	Train passage
OPENING	100	Gate opening sequence
FAULT	101	Fault state

- **State Transition Explanation:**

The controller operates as a finite state machine. After reset, it enters the IDLE state, where the gate-open command is active and road traffic is permitted. When a train approaches, the controller enters the warning state and activates the warning indication while the road signal becomes red.

After the warning counter reaches its configured threshold, the controller enters the closing state. Once the gate-closed feedback is received, the FSM moves to the train-passage state. After the train-departure input is asserted, the controller enters the opening state. When gate-open feedback is received, the controller returns to IDLE.

The design also includes a FAULT state for selected invalid input combinations. In the implemented RTL, the FAULT state is retained until reset. The controller requests gate closure and keeps the warning and road-red indications active while in this state.

- **Module:**

```
`timescale 1ns/1ps
module railway_controller #(
    parameter WARN_CYCLES = 3
)(
    input wire clk,
    input wire rst,
    input wire train_approach,
    input wire train_departed,
    input wire gate_open,
    input wire gate_closed,

    output reg gate_open_cmd,
    output reg gate_close_cmd,
    output reg warning,
    output reg road_red,
    output reg road_green
);
    localparam IDLE       = 3'b000,
               WARNING_ST = 3'b001,
               CLOSING    = 3'b010,
               TRAIN_PASS = 3'b011,
               OPENING    = 3'b100,
               FAULT      = 3'b101;

    reg [2:0] state, next_state;
    reg [7:0] warn_count;

    // State register
    always @(posedge clk or posedge rst) begin
        if (rst)
            state <= IDLE;
        else
            state <= next_state;
    end

    // Warning timer
    always @(posedge clk or posedge rst) begin
        if (rst)
            warn_count <= 0;
        else if (state != WARNING_ST)
            warn_count <= 0;
        else if (warn_count < WARN_CYCLES)
            warn_count <= warn_count + 1'b1;
    end
```

```
// Next-state logic and safety checks
  always @(*) begin
    next_state = state;
    if (gate_open && gate_closed)
      next_state = FAULT;
    else begin
      case (state)
        IDLE: begin
          if (train_departed)
            next_state = FAULT;
          else if (train_approach)
            next_state = WARNING_ST;
        end

        WARNING_ST: begin
          if (train_departed || !gate_open)
            next_state = FAULT;
          else if (warn_count >= WARN_CYCLES)
            next_state = CLOSING;
        end

        CLOSING: begin
          if (train_departed)
            next_state = FAULT;
          else if (gate_closed)
            next_state = TRAIN_PASS;
        end

        TRAIN_PASS: begin
          if (gate_open || !gate_closed)
            next_state = FAULT;
          else if (train_departed)
            next_state = OPENING;
        end

        OPENING: begin
          if (train_approach)
            next_state = FAULT;
          else if (gate_open)
            next_state = IDLE;
        end

        FAULT: begin
          next_state = FAULT;
        end
        default: next_state = FAULT;
      endcase
    end
  end
```

```
// Output logic:
    always @(*) begin
        gate_open_cmd  = 1'b0;
        gate_close_cmd = 1'b1;
        warning        = 1'b1;
        road_red       = 1'b1;
        road_green     = 1'b0;

        case (state)

            IDLE: begin
                gate_open_cmd  = 1'b1;
                gate_close_cmd = 1'b0;
                warning        = 1'b0;
                road_red       = 1'b0;
                road_green     = 1'b1;
            end

            WARNING_ST: begin
                gate_open_cmd  = 1'b1;
                gate_close_cmd = 1'b0;
            end

            CLOSING, TRAIN_PASS, FAULT: begin
                gate_open_cmd  = 1'b0;
                gate_close_cmd = 1'b1;
            end

            OPENING: begin
                gate_open_cmd  = 1'b1;
                gate_close_cmd = 1'b0;
            end

            default: begin
                gate_open_cmd  = 1'b0;
                gate_close_cmd = 1'b1;
                warning        = 1'b1;
                road_red       = 1'b1;
                road_green     = 1'b0;
            end

        endcase
    end

endmodule
```

- **Testbench:**

```
`timescale 1ns/1ps
module tb_railway;

    reg clk, rst;
    reg train_approach, train_departed;
    reg gate_open, gate_closed;

    wire gate_open_cmd, gate_close_cmd;
    wire warning, road_red, road_green;

    railway_controller #(.WARN_CYCLES(3)) uut (
        .clk(clk),
        .rst(rst),
        .train_approach(train_approach),
        .train_departed(train_departed),
        .gate_open(gate_open),
        .gate_closed(gate_closed),
        .gate_open_cmd(gate_open_cmd),
        .gate_close_cmd(gate_close_cmd),
        .warning(warning),
        .road_red(road_red),
        .road_green(road_green)
    );

    // 10 ns clock period
    always #5 clk = ~clk;
    initial begin
        $dumpfile("../VCD/railway_controller.vcd");
        $dumpvars(0, tb_railway);

        // Initial values
        clk = 0;
        rst = 1;
        train_approach = 0;
        train_departed = 0;
        gate_open = 1;
        gate_closed = 0;
        // Reset
        #12;
        rst = 0;
```

```
        // Test 1: Train approaches
        @(negedge clk);
        train_approach = 1;
        @(negedge clk);
        train_approach = 0;

       // Test 2: Warning interval
        repeat (5) @(negedge clk);

        // Test 3: Gate fully closes
        gate_open = 0;
        gate_closed = 1;

        // Test 4: Train passes
        repeat (2) @(negedge clk);

        // Test 5: Train departs
        train_departed = 1;

        @(negedge clk);
        train_departed = 0;

        // Gate begins opening
        gate_closed = 0;
        gate_open = 0;

        // Test 6: Gate fully opens
        repeat (2) @(negedge clk);
        gate_open = 1;

        // Return to IDLE
        repeat (2) @(negedge clk);

        // Test 7: Invalid sensor combination
        gate_closed = 1;

        repeat (3) @(negedge clk);

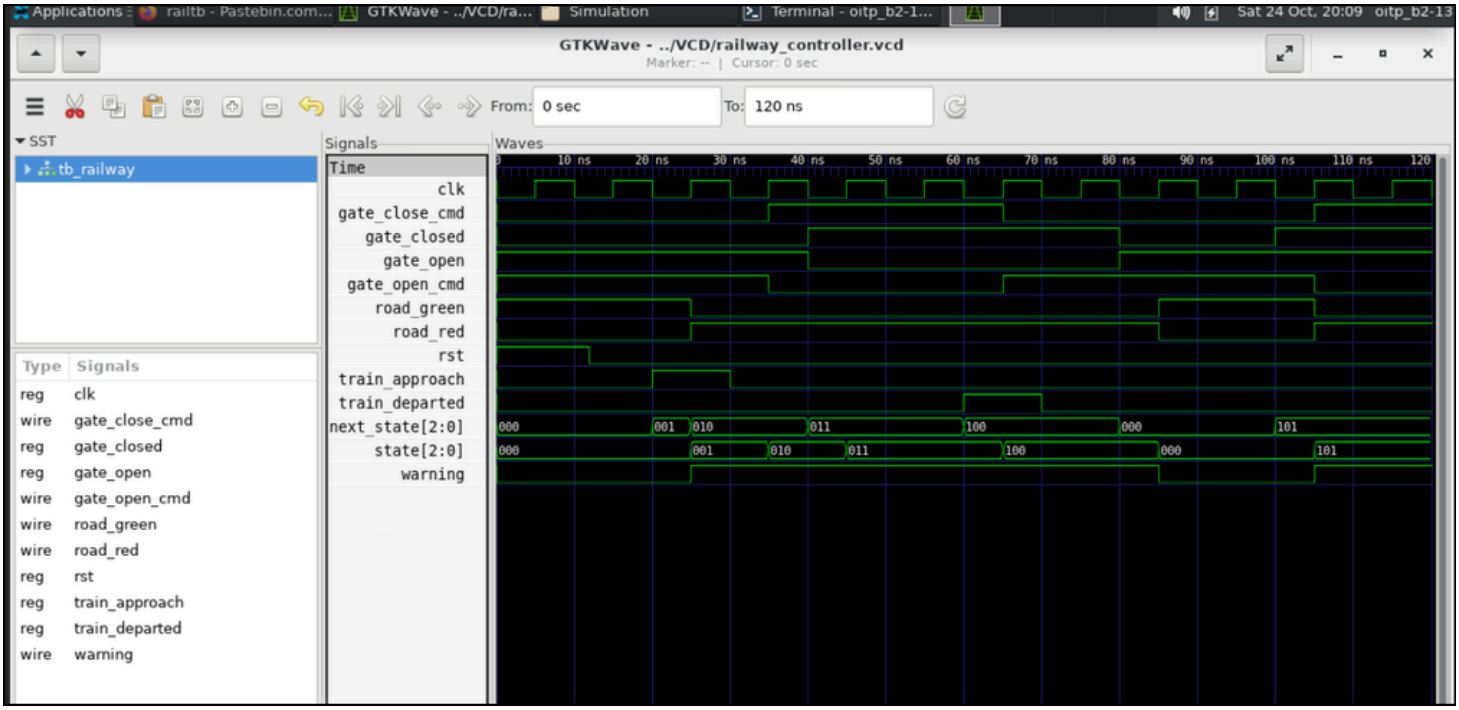
        $finish;
    end

endmodule
```

1. Simulation Commands:

- vcs -o Output/railway_controller.out ../RTL/Design/railway_controller.v ../RTL/TB/tb_railway.v
- cd Output
- ./railway_controller.out
- cd ..
- gtkwave VCD/railway_controller.vcd

• Simulation Output:



2. Coverage Commands:

- vcs -cm line+tgl+branch+cond+fsm -o Output/railway_controller.out ../RTL/Design/railway_controller.v ../RTL/TB/tb_railway.v

- cd Output
- ./railway_controller.out -cm line+tgl+branch+cond+fsm

- urg -dir railway_controller.out.vdb/ -report ../Coverage/

- **Coverage Reports:**

SYNOPSYS®

Dashboard

dashboard | **hierarchy** | **modlist** | groups | **tests** | asserts

Date: Sat Oct 24 20:19:34 2026
User: oitp_b2-13
Version: W-2024.09
Command line: urg -dir railway_controller.out.vdb -report ../Coverage/
Number of tests: 1

Total Coverage Summary

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
70.45	89.32	63.64	80.56	42.86	75.86

Hierarchical coverage data for top-level instances

SCORE	LINE	COND	TOGGLE	FSM	BRANCH	NAME
70.45	89.32	63.64	80.56	42.86	75.86	tb_railway

Total Module Definition Coverage Summary

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
70.45	89.32	63.64	80.56	42.86	75.86

SYNOPSYS®

Design Module List

dashboard | **hierarchy** | **modlist** | groups | **tests** | asserts

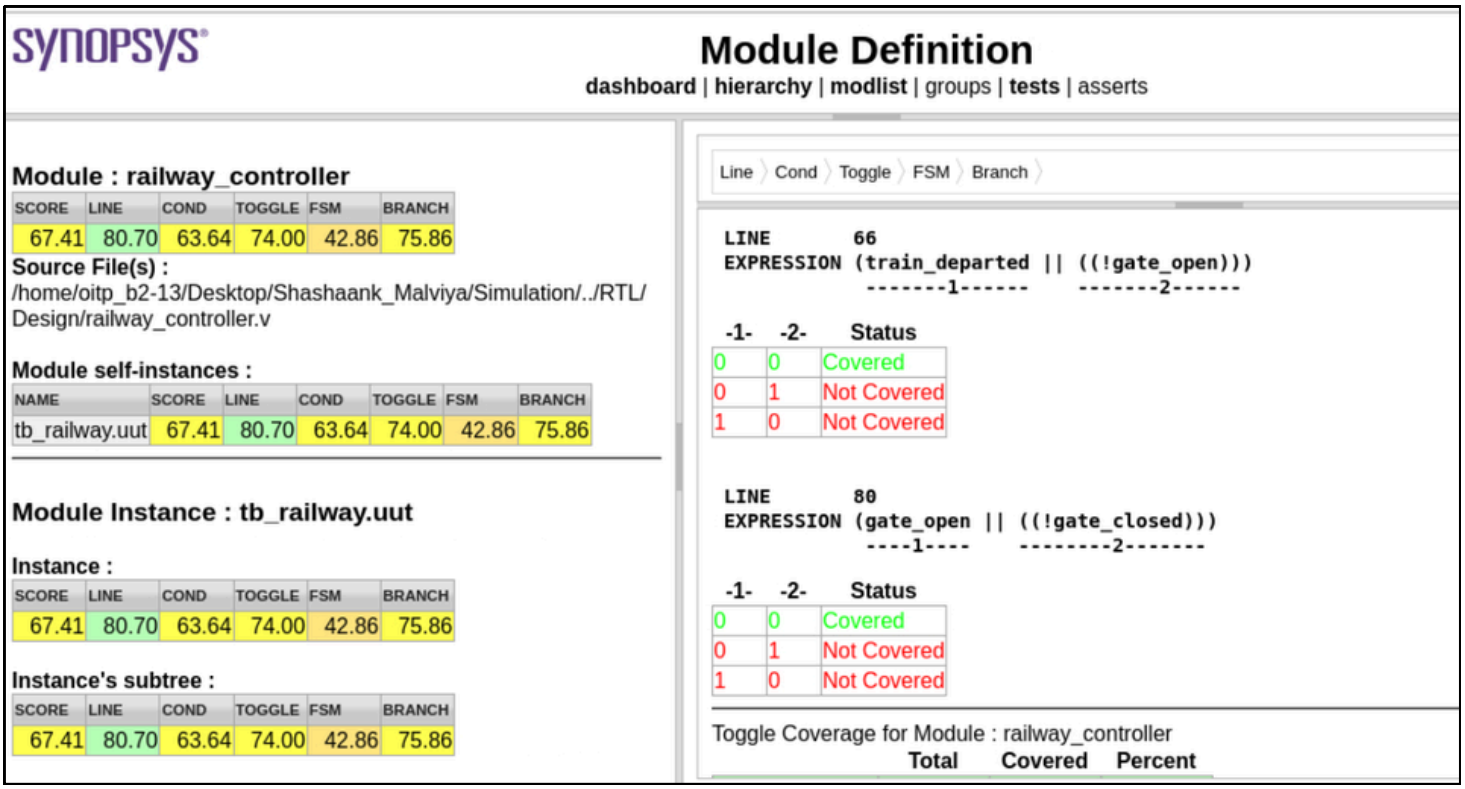
Total Module Definition Coverage Summary

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
70.45	89.32	63.64	80.56	42.86	75.86

Total modules in report: 2

Expand All | Collapse All

NAME	SCORE	LINE	COND	TOGGLE	FSM	BRANCH
railway_controller	67.41	80.70	63.64	74.00	42.86	75.86
tb_railway	97.73	100.00		95.45		



Metric	Coverage Achieved
Line Coverage	89.32
Toggle Coverage	80.56
Condition coverage	63.64
Branch coverage	75.86
FSM coverage	42.86

- ## Synthesis Constraints File:

create_clock -name clk -period 10 [get_ports clk]

set_input_delay 1.0 -clock clk [get_ports {train_approach train_departed gate_open gate_closed}]

set_output_delay 1.0 -clock clk [get_ports {gate_open_cmd gate_close_cmd warning road_red road_green}]

set_false_path -from [get_ports rst]

- ## Synthesis File:

```
source /home/oitp_b2-13/Desktop/Shashaank_Malviya/Scripts/dc_setup.tcl

set TOP railway_controller

read_verilog /home/oitp_b2-13/Desktop/Shashaank_Malviya/RTL/Design/${TOP}.v

current_design $TOP
link

source /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Constraints/${TOP}.sdc

check_design

compile_ultra

report_qor > /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Reports/${TOP}_qor.rpt
report_area > /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Reports/${TOP}_area.rpt
report_timing > /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Reports/${TOP}_timing.rpt
report_power > /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Reports/${TOP}_power.rpt

write -format verilog -hierarchy
-output /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Netlist/${TOP}_synth_netlist.v

write_sdc /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Netlist/${TOP}_output.sdc

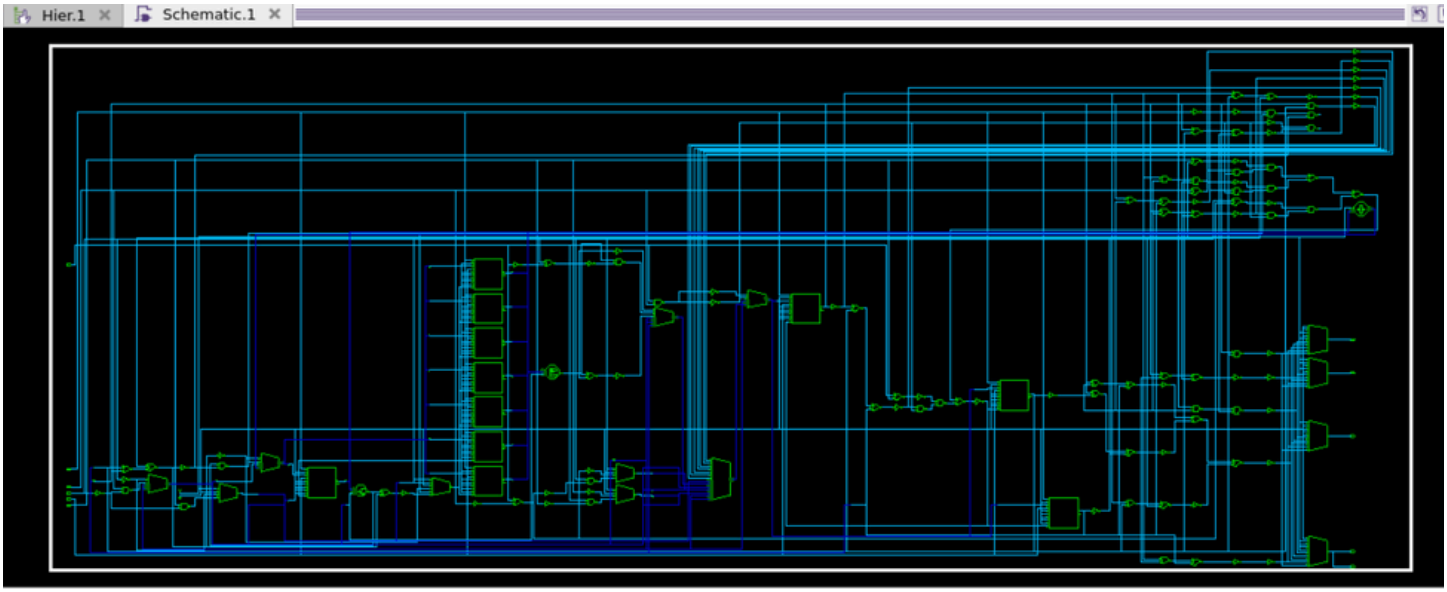
write_file -format ddc -hierarchy
-output /home/oitp_b2-13/Desktop/Shashaank_Malviya/Synthesis/Backup/${TOP}.ddc
```

3. Synthesis Commands:

- dc_shell
- source synth.tcl
- gui_show

- **Synthesized Schematic:**





- **Synthesized Netlist:**

~/Desktop/Shashaank_Malviya/Synthesis/Netlist/railway_controller_synth_netlist.v - Mousepad

File Edit Search View Document Help

```
////////////////////////////////////////////////////////////
// Created by: Synopsys Design Compiler(R)
// Version   : W-2024.09
// Date      : Sat Oct 24 20:28:14 2026
////////////////////////////////////////////////////////////


module railway_controller ( clk, rst, train_approach, train_departed,
        gate_open, gate_closed, gate_open_cmd, gate_close_cmd, warning,
        road_red, road_green );
  input clk, rst, train_approach, train_departed, gate_open, gate_closed;
  output gate_open_cmd, gate_close_cmd, warning, road_red, road_green;
  wire   N0, N1, N2, N3, N4, N5, N6, N7, N8, N9, N10, N11, N12, N13, N14, N15,
         N16, N17, warning, N18, N19, N20, N21, N22, N23, N24, N25, N26, N27,
         N28, N29, N30, N31, N32, N33, N34, N35, N36, N37, N38, N39, N40, N41,
         N42, N43, N44, N45, N46, N47, N48, N49, N50, N51, N52, N53, N54, N55,
         N56, N57, N58, N59, N60, N61, N62, N63, N64, N65, N66, N67, N68, N69,
         N70, N71, N72, N73, N74, N75, N76, N77, N78, N79, N80, N81, N82, N83,
         N84, N85, N86, N87, N88, N89, N90, N91, N92, N93, N94, N95, N96, N97,
         N98, N99, N100, N101, N102, N103, N104, N105, N106, N107, N108, N109,
         N110, N111, N112, N113, N114, N115, N116, N117, N118, N119, N120,
         N121, N122, N123, N124, N125, N126, N127, N128, N129, N130, N131,
         N132, N133, N134, N135, net53, net54;
  wire   [2:0] state;
  wire   [2:0] next_state;
  wire   [7:0] warn_count;
  assign road_red = warning;

  \**SEQGEN**  \state_reg[2]  ( .clear(rst), .preset(1'b0), .next_state(
        next_state[2]), .clocked_on(clk), .data_in(1'b0), .enable(1'b0), .Q(
        state[2]), .synch_clear(1'b0), .synch_preset(1'b0), .synch_toggle(1'b0), .synch_enable(N126) );
  \**SEQGEN**  \state_reg[1]  ( .clear(rst), .preset(1'b0), .next_state(
        next_state[1]), .clocked_on(clk), .data_in(1'b0), .enable(1'b0), .Q(
        state[1]), .synch_clear(1'b0), .synch_preset(1'b0), .synch_toggle(1'b0), .synch_enable(N126) );
```

- **Synthesis reports**

A. Quality report

~/Desktop/Shashaank_Malviya/Synthesis/Reports/railway_controller_qor.rpt

File Edit Search View Document Help

```
Report : qor
Design : railway_controller
Version: W-2024.09
Date   : Sat Oct 24 20:28:14 2026
****************************************


  Timing Path Group 'clk'
  -----------------------------------
  Levels of Logic:               3.00
  Critical Path Length:          0.00
  Critical Path Slack:           9.00
  Critical Path Clk Period:     10.00
  Total Negative Slack:          0.00
  No. of Violating Paths:        0.00
  Worst Hold Violation:          0.00
  Total Hold Violation:          0.00
  No. of Hold Violations:        0.00
  -----------------------------------


  Cell Count
  -----------------------------------
  Hierarchical Cell Count:          6
  Hierarchical Port Count:         95
  Leaf Cell Count:                210
  Buf/Inv Cell Count:              52
  Buf Cell Count:                  20
  Inv Cell Count:                  32
  CT Buf/Inv Cell Count:            0
  Combinational Cell Count:       199
  Sequential Cell Count:           11
  Macro Count:                      0
  -----------------------------------
```

~/Desktop/Shashaank_Malviya/Synthesis/Reports/railway_controller_qor.rpt - N

File Edit Search View Document Help

```
Area
---------------------------------
Combinational Area:        0.000000
Noncombinational Area:     0.000000
Buf/Inv Area:              0.000000
Total Buffer Area:             0.00
Total Inverter Area:           0.00
Macro/Black Box Area:      0.000000
Net Area:                  0.000000
---------------------------------
Cell Area:                 0.000000
Design Area:               0.000000


Design Rules
---------------------------------
Total Number of Nets:           239
Nets With Violations:             0
Max Trans Violations:             0
Max Cap Violations:               0
---------------------------------


Hostname: smartlab6.calicut.nielit.in

Compile CPU Statistics
---------------------------------------
Resource Sharing:                   0.00
Logic Optimization:                 0.00
Mapping Optimization:               0.00
---------------------------------------
Overall Compile Time:               0.00
Overall Compile Wall Clock Time:    0.00
```

```
--------------------------------------------------------------------

Design  WNS: 0.00  TNS: 0.00  Number of Violating Paths: 0


Design (Hold)  WNS: 0.00  TNS: 0.00  Number of Violating Paths: 0

--------------------------------------------------------------------
```

B. Timing report

~/Desktop/Shashaank_Malviya/Synthesis/Reports/railway_controller_timing.rpt - N

le Edit Search View Document Help

```
****************************************

perating Conditions: nom_pvt   Library: dw_foundation.sldb
re Load Model Mode: top

Startpoint: state_reg[2]
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: gate_close_cmd
          (output port clocked by clk)
Path Group: clk
Path Type: max

Point                                                   Incr       Path
--------------------------------------------------------------------------
clock clk (rise edge)                                   0.00       0.00
clock network delay (ideal)                             0.00       0.00
state_reg[2]/clocked_on (**SEQGEN**)                    0.00       0.00 r
state_reg[2]/Q (**SEQGEN**)                             0.00       0.00 r
C221/Z (GTECH_AND2)                                     0.00       0.00 r
B_17/Z (GTECH_BUF)                                      0.00       0.00 r
C295/Z_0 (*SELECT_OP_5.1_5.1_1)                         0.00       0.00 r
gate_close_cmd (out)                                    0.00       0.00 r
data arrival time                                                  0.00

clock clk (rise edge)                                  10.00      10.00
clock network delay (ideal)                             0.00      10.00
output external delay                                  -1.00       9.00
data required time                                                 9.00
--------------------------------------------------------------------------
data required time                                                 9.00
data arrival time                                                  0.00
--------------------------------------------------------------------------
slack (MET)                                                        9.00
```

C. Power report

~/Desktop/Shashaank_Malviya/Synthesis/Reports/railway_controller_power.rpt - Mousepad

File Edit Search View Document Help

```
    Capacitance Units = Unitless
    Time Units = Unitless
    Dynamic Power Units = Unitless    (derived from V,C,T units)
    Leakage Power Units = Unitless


Attributes
----------
i - Including register clock pin internal power


  Cell Internal Power  =   0.0000
  Net Switching Power  =   0.0000
                         ---------
Total Dynamic Power    =   0.0000

Cell Leakage Power     =   0.0000

Error: Either dynamic power or leakage power, in the library, is unitless. Unable to display complete power group summary

                 Internal         Switching           Leakage            Total
Power Group      Power            Power               Power              Power   (   %    )  Attrs
--------------------------------------------------------------------------------------------------
io_pad             0.0000           0.0000              0.0000               NA   (     N/A)
memory             0.0000           0.0000              0.0000               NA   (     N/A)
black_box          0.0000           0.0000              0.0000               NA   (     N/A)
clock_network      0.0000           0.0000              0.0000               NA   (     N/A)  i
register           0.0000           0.0000              0.0000               NA   (     N/A)
sequential         0.0000           0.0000              0.0000               NA   (     N/A)
combinational      0.0000           0.0000              0.0000               NA   (     N/A)
--------------------------------------------------------------------------------------------------
Total              0.0000           0.0000              0.0000               NA
1
```

• Conclusion:

The Automatic Railway Level Crossing Controller was successfully designed and implemented using Verilog Hardware Description Language. The controller uses a finite state machine to coordinate train detection, gate operation, warning indication, and road traffic signals. It also incorporates a warning counter and fault-detection logic to handle selected invalid sensor conditions.

Functional verification was performed using Synopsys VCS, and the resulting signal waveforms were analyzed using GTKWave. The simulation exercised the normal crossing sequence, including train approach, warning, gate closing, train passage, gate opening, and return to the idle state. A selected invalid gate-sensor combination was also applied to evaluate the fault-handling behavior.

Code coverage analysis was performed using line, toggle, condition, branch, and FSM coverage metrics. The results showed 89.32% line coverage, 80.56% toggle coverage, 63.64% condition coverage, 75.86% branch coverage, and 42.86% FSM coverage. These results demonstrate that the testbench exercised several important parts of the design while also identifying opportunities for additional verification.

The RTL design was subsequently synthesized using Synopsys Design Compiler with the configured SAED32 standard-cell library. The synthesis process generated a technology-mapped netlist, a schematic representation, and reports for quality of results, area, timing, and power.

Overall, the project demonstrates an RTL-to-synthesis workflow for an FSM-based digital control system. It provides practical experience in Verilog design, simulation, waveform analysis, coverage evaluation, timing constraints, and logic synthesis. Further verification and confirmation of the numerical synthesis results would strengthen the assessment of the implementation.